Assignment-2 for the post of Web & Tech Manager, E-Cell IIT Bombay

---

Submission Deadline: 16th March 2026, 06:00 AM ***Deadline is strict.***

You are advised to complete as much of the assignment as possible. Focus on implementing the core functionality first, and then attempt additional improvements for bonus points.

Use of AI tools is allowed. However, you are expected to fully understand every part of the code you submit. During evaluation, you may be asked to explain your implementation.

Problem Statement

E-Cell IIT Bombay runs multiple initiatives throughout the year, including events, competitions, partnerships, marketing campaigns, media releases, and internal operations. These initiatives rely on several digital platforms such as websites, dashboards, and internal tools.

Currently, most issues and queries are reported informally through personal messages, emails, or direct communication with team members. As the organization grows, this approach becomes difficult to manage.

Some of the common challenges include:

- **Queries being sent to the wrong person or department**
- **Lack of visibility into the progress of ongoing issues**

- Duplicate reports of the same problem
- No centralized record of past queries or their resolutions
- Difficulty coordinating between managers and their teams

To address these challenges, the Web & Tech team plans to build a centralized portal to manage queries and issues across the organization.

The system should allow users to raise queries and allow internal teams to collaboratively resolve them.

The portal must support multiple levels of access, since different members of the organization should only see the queries relevant to them.

For example:

- Some users may have visibility across the entire system
- Some users may manage queries related to a specific initiative or domain
- Some users may only be able to work on queries assigned to them

Additionally, queries may involve discussion between multiple stakeholders, attachments such as screenshots or files, and a clear history of updates.

Your Task

Design and implement a Query and Issue Management Portal that addresses the above problem.

Your solution should demonstrate:

- strong system design thinking

- good engineering practices ● attention to product quality

You are free to choose appropriate technologies, architectures, and design patterns.

Part 1 - System Design

Explain how you would design the system.

Your explanation may include (but is not limited to):

- how queries should be represented and organized
- how different users interact with the system
- how access control should work
- how queries should be assigned or routed
- how conversations or updates within a query should be handled
- how attachments such as screenshots or files should be stored

Clearly justify your design choices.

Part 2 - Access Control

The portal must support multiple levels of access and responsibility.

Design a permission system that enables:

- hierarchical access
- controlled visibility of queries

- delegation of responsibilities between users

Explain how this model would function in practice and how unauthorized access will be prevented.

Part 3 - Implementation

Build a working prototype of the portal.

Your implementation should include:

- a user interface for creating and managing queries
- a backend system to manage users and queries
- authentication and authorization
- the ability to attach screenshots or files
- a mechanism for updating or discussing queries

Allowed Technology Stacks

Frontend

- Angular
- React

Backend

- Django
- Node.js (Express or NestJS)

Database

- **Any database of your choice (Postgres/mySQL/MongoDB) Product Quality Requirements**

Your implementation must meet the following standards:

- **The platform should look professional and production-ready**
- **The UI should not resemble generic AI-generated interfaces**
- **The interface must be fully responsive across devices and screen sizes**
- **Proper error handling must be implemented on both frontend and backend**
- **The application should be stable and free from obvious bugs**

Attention to UI/UX quality, usability, and robustness will be considered during evaluation.

Engineering Understanding Requirement

You are expected to fully understand every part of the system you implement.

During evaluation, you may be asked to explain:

- **how different parts of your system work**
- **why you chose specific design decisions**
- **how your code handles edge cases or failures**
- **how different components interact with each other**

You should not rely on code that you cannot explain or justify.

Part 4 - Additional Feature

Propose and implement at least one feature that significantly improves the usefulness of the portal.

This feature should demonstrate thoughtful engineering and add meaningful value to the system.

Explain:

- **what the feature does**
- **why it improves the system**
- **how it is implemented**

Part 1 - System Design:

1. **How Queries Are Represented and Organised**
   Each query is a single MongoDB document using an embedded model — comments, attachments, and status history live inside the query rather than in separate collections. One read returns everything needed to render the full detail page, with no joins. Key fields: title, description, category (tech/events/marketing etc.), status (open→in_progress→pending_info→resolved→closed), priority, createdBy, assignedTo[] (array for multi-manager support), comments[], attachments[], and statusHistory[] (append-only audit log).

2. **How Different Users Interact Member:**
   User:Submits queries, views only their own queries, adds public comments. Cannot change status or see internal notes.
   Manager: Sees all queries in their domain plus any explicitly assigned to them. Can update status/priority, post internal notes (hidden from members), and assign queries to other managers.

Admin: Full visibility across all domains. Can assign queries to multiple managers, delete queries, and manage user roles and domains.

3. **How Access Control Works**
   Authentication: JWT tokens signed on login, stored client-side, sent as Bearer on every request. Passwords hashed with bcrypt (12 rounds). Expired tokens return 401 and redirect to login.
   Authorisation: Every protected route runs a protect middleware (verifies JWT) then restrictTo() (checks role). Query visibility is enforced via a buildVisibilityFilter(user) function that returns a different MongoDB filter per role — admins get {}, managers get their domain filter, members get an $or on their own IDs. Internal notes are stripped server-side before the response reaches members. All role promotions require an existing admin — self-registration always creates a member.

4. **How Queries Are Assigned and Routed**
   • Auto-assignment: On submission, the system finds all active managers whose domain matches the query category and adds them all to assignedTo[]. Each gets a notification instantly.
   • Manual assignment: Admins and Managers can tick/untick managers via a checkbox UI on the query detail page. Cross-domain assignment is supported — a tech query can be assigned to an events manager if needed.
   • Newly assigned managers gain visibility even outside their domain. Removing them revokes access unless the query is in their domain
5. **How Conversations and Updates Are Handled**
   • Comments are embedded in the query document. Public comments are visible to all parties; internal notes (isInternal: true) are visible only to managers and admins — enforced server-side.
   • Status changes append to statusHistory with actor, timestamp, and an optional note — giving a full audit trail.
   • Notifications fire automatically on key events (query created, status changed, comment added, assignment changed) and appear in a bell icon that polls every 30 seconds.

6. **How Attachments Are Stored**
   • Files are handled by Multer and stored on the server filesystem with a random 32-char hex filename to prevent collisions and enumeration.
   • Metadata (original name, MIME type, size, URL) is stored in the query's attachments[] array. Files are served as Express static assets.
   • Validation enforces a 10MB per-file limit, max 5 files, and a MIME whitelist (images, PDF, DOC, TXT).
   • Production path: swap Multer's disk storage for AWS S3 or Cloudinary — the metadata schema and API contract stay identical

Part 2 – Access Control

1. **Hierarchical Access**
   The system uses a three-tier Role-Based Access Control (RBAC) model. Each role inherits the capabilities of the one below it and adds more.

Role	View Queries	Edit Queries	Assign Managers	Manage Users	Delete
Admin	All domains	Any query	✓	✓	✓
Manager	Own domain + assigned	Own domain + assigned	✓	×	×
Member	Own queries only	None	✓	×	×

- **Member Actions:** A Member submits a query and can see only their own submissions.
- **Admin Oversight:** Admins have oversight of all system activities.
- **Role Limitations:** No role can elevate its own permissions.
- **Promotion:** Only an Admin has the authority to promote a Member to a Manager.

2. Controlled Visibility of Queries

Visibility is enforced at the database level using the buildVisibilityFilter(user) function, which applies specific MongoDB filters based on the user's role.

- **Admin Filter:** No filter is applied; Admins can see everything.

- **Manager Filter:** Filters for the user's specific domain or queries explicitly assigned to them across domains.
- **Member Filter:** Filters for queries created by the user.
- **Security Enforcement:** Filters are re-validated on every individual query fetch.
- **URL Protection:** Direct URL access to unauthorized queries returns a **403 Forbidden** error rather than just hiding links.
- **Internal Notes:** Comments marked as isInternal are stripped server-side; members inspecting API responses cannot see this content.

3. Delegation of Responsibilities

The system supports both automatic and manual delegation to streamline query management.

Automatic Domain Routing

- Submitted queries are automatically assigned to all active managers in that matching category.
- Assigned managers receive notifications and immediate edit access.
- This process is designed to remove the need for manual routing in structured teams.

Manual & Role Delegation

- **Cross-Domain Assignment:** Admins and Managers can manually assign queries to any manager, granting them full edit access regardless of their primary domain.
- **Revoking Access:** Removing a manager from the assignment immediately revokes their access.
- **User Management:** Admins can change user roles and domains via the Users page.
- **Inheritance:** Newly promoted managers immediately inherit visibility of all existing queries in their assigned domain.
- **Lockout Prevention:** Admins are prohibited from demoting themselves to prevent accidental lockouts.

4. How Unauthorized Access Is Prevented

Security is maintained through multiple layers, ensuring the frontend is never the final security boundary.

Authentication Layer

- **Protect Middleware:** Every API route (excluding login and register) verifies JWT signatures and expiry.
- **Password Security:** Passwords are hashed using **bcrypt** (12 salt rounds).
- **Data Privacy:** Plaintext passwords are never stored. The password field is excluded from database queries by default via the Mongoose schema.
- **JWT Integrity:** The JWT secret is stored in environment variables and never exposed to the client.

Authorization Layer

- **Sensitive Routes:** The restrictTo(...roles) middleware blocks unauthorized roles before any business logic executes.
- **Access Validation:** assertCanAccess() prevents direct URL bypass by re-checking individual query permissions.
- **Edit Protection:** assertCanManage() ensures only admins or assigned managers can PATCH (edit) a query.
- **Role Integrity:** All role promotions require an admin's request. Self-registration is strictly locked to the "Member" role; role fields in registration requests are ignored.
- **Server-Side Logic:** All permission checks are performed on the backend based on server-side records, not client-provided input.

Part 4 - Additional Feature

E-Cell Query Portal - Additional Feature

Feature: Real-Time Notification System

The system implements a persistent, role-aware notification system designed to proactively alert users about relevant events, removing the need for them to manually check the portal for updates.

1. What the Feature Does

- **User Interface**: A bell icon in the top navigation bar displays a badge with the unread notification count.
- **Interaction**: Clicking the icon opens a dropdown panel showing recent notifications, including type icons, messages, timestamps, and read/unread status.
- **Automatic Triggers**: Notifications are generated automatically during four key events:

Event	Who is Notified
New query submitted	All active admins and managers
Query assigned to a manager	The assigned manager(s)
Query status changed	The query creator,all assignees and admin
New comment added	Query creator + all assignees (public comments only)

- **Functionality**: Each notification includes a type, title, message, query reference, and a read boolean.
- **Navigation**: Clicking a notification marks it as read and navigates the user directly to the related query.
- **Bulk Actions**: A "Mark all read" option is available to clear the unread badge in one action.

2. Why It Improves the System

- **Push-Based Efficiency**: Moves the portal from a pull-based system (where users must remember to log in) to a push-based system.
- **Accountability**: Addresses issues found in informal channels like WhatsApp or email, where messages are often missed and no accountability trail exists.
- **Faster Response**: Managers are alerted immediately when a query enters their domain, significantly dropping response times.
- **Transparency**: Members no longer have to guess the status of their queries.
- **Engagement**: The unread badge acts as a natural prompt for users to return to the portal.

3. How It Is Implemented

Backend Architecture

- **Data Model**: Stores the recipient (User reference), type, title, message, query relation, read status, and timestamps.
- **Service Layer**: NotificationService.create() is a lightweight helper called within the QueryService at specific event points.
- **Non-Blocking**: Notification creation is wrapped in try/catch blocks; a failure in the notification database write will never cause a 500 error or break the main query request flow.
- **API Endpoints**: Includes GET /api/notifications (fetch and count), PATCH /api/notifications/:id/read, and PATCH /api/notifications/read-all.

- **Performance**: A MongoDB index on recipient, read, and createdAt ensures fast lookups as the data grows.

Frontend Logic

- **Polling Strategy**: The system fetches updates every 30 seconds via setInterval, providing a reliable solution without requiring WebSocket infrastructure.
- **Visuals**: An amber badge shows the unread count, capped at "9+".
- **Persistence**: The component is mounted in AppLayout so it persists across all pages without remounting.
- **UX Details**: A "click outside" handler automatically closes the dropdown using a useRef and mousedown listener.

Security and Logic Rules

- **Role-Awareness**: Internal comments do not trigger notifications for members; the isInternal flag gates notification creation.
- **Noise Reduction**: Users do not receive notifications for their own actions (e.g., commenting on their own query).
- **Scalability**: The architecture is designed so that polling can be upgraded to WebSockets or Server-Sent Events in the future without changing the database schema.